

# Generative Diffusion Model Driven Massive Random Access in Massive MIMO Systems

Keke Ying, Zhen Gao, *Member, IEEE*, Sheng Chen, *Life Fellow, IEEE*, Tony Q.S. Quek, *Fellow, IEEE*, and H. Vincent Poor, *Life Fellow, IEEE*

**Abstract**—Massive random access is an important technology for achieving ultra-massive connectivity in next-generation wireless communication systems. It aims to address key challenges during the initial access phase, including active user detection (AUD), channel estimation (CE), and data detection (DD). This paper examines massive access in massive multiple-input multiple-output (MIMO) systems, where deep learning is used to tackle the challenging AUD, CE, and DD functions. First, we introduce a Transformer-AUD scheme tailored for variable pilot-length access. This approach integrates pilot length information and a spatial correlation module into a Transformer-based detector, enabling a single model to generalize across various pilot lengths and antenna numbers. Next, we propose a generative diffusion model (GDM)-driven iterative CE and DD framework. The GDM employs a score function to capture the posterior distributions of massive MIMO channels and data symbols. Part of the score function is learned from the channel dataset via neural networks, while the remaining score component is derived in a closed form by applying the symbol prior constellation distribution and known transmission model. Utilizing these posterior scores, we design an asynchronous alternating CE and DD framework that employs a predictor-corrector sampling technique to iteratively generate channel estimation and data detection results during the reverse diffusion process. Simulation results demonstrate that our proposed approaches significantly outperform baseline methods with respect to AUD, CE, and DD.

**Index Terms**—Deep learning, massive MIMO, massive random access, Transformer, generative models.

## I. INTRODUCTION

IN wireless communications, multi-user random access is a critical step in establishing initial communication links. This process involves techniques such as activity user detection (AUD), channel estimation (CE), and multi-user data detection (DD). Traditional cellular networks often use scheduling-based access protocols and orthogonal multiple access methods [1]. As networks evolve, the 6G vision outlines more stringent performance requirements for scenarios involving massive communication. This vision necessitates a connection density

ranging from  $10^6$  to  $10^8$  devices per square kilometer [2]. As the number of users increases, dependence on orthogonal multiple access will lead to excessive overhead, substantially affecting communication efficiency.

To overcome the limitations of existing scheduling-based random access protocols, the grant-free non-orthogonal multiple access scheme has been proposed [1]. By eliminating the need for complex handshaking between users and the base station (BS), terminals can directly transmit uplink signals composed of non-orthogonal preamble sequences and payload data. This approach effectively reduces communication delays while enabling more devices to access the network simultaneously. However, it also increases the complexity of designing receiver detection algorithms. Researchers have developed various advanced receiver algorithms, which can be applied to one or more tasks in AUD, CE, and DD, depending on the specific challenges considered.

## A. Related Work

For massive random access, identifying the set of active users is a prerequisite for executing subsequent transmission tasks. Studies in [3]–[5] have developed a covariance-based AUD method. This approach initially derives the likelihood function of user activity from received signals using a transmission model, framing AUD as a maximum likelihood estimation problem. The coordinate descent method is then employed to iteratively estimate each user’s activity. However, the algorithm’s computational complexity is high due to the need for computing the inverse of the covariance matrix in each iteration and performing serial iterative estimation for multiple users. To address this challenge, the authors of [6], [7] proposed advanced neural network-based AUD methods, which adopt binary cross-entropy as the loss function and utilize a heterogeneous Transformer to directly map received signals to the active user support set, achieving superior active user detection performance and low inference latency.

The aforementioned research assumes that the channel between users and the BS is characterized by a multi-antenna Rayleigh channel, and it does not tackle the scenarios of non-Rayleigh channels. In non-Rayleigh environments, a common strategy involves the joint execution of AUD and CE tasks. This is often framed as a compressed sensing (CS) problem, which is solved using traditional greedy algorithms, Bayesian approaches, or deep learning methods. For example, the existing algorithms, such as orthogonal matching pursuit (OMP) [8], approximate message passing (AMP) [9], and orthogonal AMP (OAMP) [10], exploit the inherent sparsity of user activity to recover the user channel matrix while simultaneously identifying active users. To reduce computational complexity and enhance overall performance, model-driven deep learning

Keke Ying is with the School of Information and Electronics, Beijing Institute of Technology, Beijing 100081, China (e-mail: ykk@bit.edu.cn).

Zhen Gao (*corresponding author*) is with State Key Laboratory of CNS/ATM, Beijing Institute of Technology (BIT), Beijing 100081, China, also with BIT Zhuhai 519088, China, also with the MIT Key Laboratory of Complex-Field Intelligent Sensing, BIT, Beijing 100081, China, also with the Advanced Technology Research Institute of BIT (Jinan), Jinan 250307, China, and also with the Yangtze Delta Region Academy, BIT (Jiaxing), Jiaxing 314019, China (e-mail: gaozhen16@bit.edu.cn).

Sheng Chen is with the School of Computer Science and Technology, Ocean University of China, Qingdao 266100, China (e-mail: sqc@ecs.soton.ac.uk).

T. Q. S. Quek is with the Singapore University of Technology and Design, Singapore 487372, and also with the Yonsei Frontier Lab, Yonsei University, South Korea (e-mail: tonyquek@sutd.edu.sg).

H. Vincent Poor is with the Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ 08544 USA (e-mail: poor@princeton.edu).

approaches [11], [12] have been developed based on traditional CS algorithm frameworks. By integrating learnable parameters with the conventional backbones, such networks demonstrate superior performance under low pilot overhead conditions while significantly accelerating convergence. However, since the primary aim of CS is CE, with activity detection as a secondary outcome, joint AUD and CE methods typically necessitate higher pilot overhead than covariance-based techniques, which directly target the active user set detection [3].

Once the AUD and CE processes are completed, the massive MIMO DD problem at BS often becomes overdetermined. Various advanced detection algorithms have been developed to address this challenge. In addition to traditional methods like least squares (LS) and linear minimum mean square error (LMMSE), algorithms such as OAMP [13], expectation propagation (EP) [14], and deep learning-based techniques [15]–[18] have demonstrated excellent performance. However, these DD methods rely on the channel state information (CSI) estimated in the earlier stage, and errors in CSI estimation will degrade the detection performance.

Performing joint CE and DD (JCEDD) can enhance overall achievable performance. The work [16] presents an iterative JCEDD scheme that accounts for channel estimation errors during data detection and employs a model-driven OAMP unfolding algorithm to improve detection. In the CE phase, demodulated data are used as pilots to augment LMMSE CE, resulting in superior performance. Additionally, message-passing algorithms have been developed for the joint AUD, CE, and DD [19], [20]. Despite their good performance, these algorithms necessitate intricate message-passing processes, with convergence and performance heavily reliant on initial estimates and the alignment between the assumed prior distribution and the underlying system's probability distribution, thereby limiting their applicability.

In recent years, generative model-based methods have been extensively applied to address inverse problems in communications. Unlike Bayesian algorithms, which require manual assumptions of prior signal distributions, generative models employ neural networks to model the signal prior, offering a more effective representation of complex signal distributions. As detailed in [21], the score matching with Langevin dynamics (SMLD) framework is used to tackle the CE problem. This approach involves estimating the score function, defined as the gradient of a log-probability density function [22]. By combining the channel prior's score function, learned from a neural network, with the likelihood score derived from observations, the posterior estimate of high-dimensional channels can be incrementally sampled from Gaussian noise using annealed Langevin dynamics (ALD) [22], [23]. The SMLD framework can also address the MIMO detection problem [24]–[26]. To handle the non-differentiable symbol constellation prior, a noise-perturbed version of the symbol's distribution serves as a proxy function to derive the score function [25]. Furthermore, the SMLD framework has been effectively applied to the JCEDD problem [27] as well as the quantized CE problem [28], demonstrating improved performance over existing deep learning and traditional methods.

Another category of generative model frameworks is the

denoising diffusion probabilistic model (DDPM) [29]. DDPM methods have been widely applied to denoising tasks in communication systems. For example, the study [30] proposed a lightweight diffusion model that first obtains an initial CE using LS and then progressively denoises it to enhance CE performance. Additionally, the work [31] applied diffusion models to denoise semantic features at the receiver, thereby improving semantic transmission performance.

It has been indicated in [32] that the DDPM process implicitly calculates score function for each noise level, and both DDPM and SMLD can be unified under the framework of score-based generative models. These models can be described using stochastic differential equations (SDEs), which model both the forward noise-injection and reverse generation processes. This SDE approach offers greater flexibility in sampling algorithms, resulting in improved performance compared to existing generative models.

## B. Our Contributions

This paper proposes a new two-stage algorithm to solve the AUD, CE, and DD tasks in massive random access systems. The first stage identifies the active users set (AUS) through a Transformer-based detector, while the second stage utilizes generative diffusion models (GDMs) to perform JCEDD for active user equipment (UEs). By contrast, traditional approaches based on CS first explicitly estimate the channels and then perform AUD and DD, which degrades dramatically in the case of insufficient pilots, while covariance-based approaches only consider AUD and neglect CE and DD. Our main contributions are summarized as follows.

- We design a variable pilot length AUD network (VPL-AUDNet) for AUD task. Specifically: 1) We propose the variable pilot length transmission framework and corresponding adaption mechanism that allows a single network to be applicable to different pilot lengths without retraining. 2) We introduce a spatial correlation mechanism to extract spatial correlation features from the received signals, thereby improving network-based AUD performance in complex correlated channels.
- We model the CE and DD tasks as reverse diffusion processes that sample from a joint distribution. These processes gradually transform initial noise into the original channel and data symbols. By employing advanced predictor-corrector samplers, our scheme outperforms existing JCEDD schemes.
- To accommodate the distinct distribution characteristics of unknown continuous channel distributions and known discrete data constellation distributions, we propose an asynchronous iterative CE and DD framework. This framework integrates an LMMSE-based CE initialization method, which reduces the number of sampling steps and enhances convergence performance.

The remainder of this paper is organized as follows. Section II briefly introduces the massive access system model, and Section III formulates the problems of AUD and JCEDD. Section IV details the proposed Transformer-based AUD scheme, while Section V elaborates on our generative models-based JCEDD

scheme. Section VI presents the simulation results, and Section VII concludes the paper.

*Notations:* Matrices and column vectors are denoted by uppercase and lowercase boldface letters, respectively.  $\mathbf{I}_N$  is the  $N \times N$  identity matrix,  $\mathbf{0}_{N \times M}$  is the  $N \times M$  all-zero matrix, and  $\mathbf{1}_N$  is the  $N \times 1$  all-one vector.  $(\cdot)^T$ ,  $(\cdot)^*$ ,  $(\cdot)^H$ ,  $(\cdot)^{-1}$ , and  $\mathbb{E}\{\cdot\}$  denote the transpose, conjugate, Hermitian transpose, inversion, and expectation operations, respectively.  $|\mathcal{A}|_c$  indicates the cardinality of the set  $\mathcal{A}$ .  $\text{diag}(\mathbf{a})$  transforms vector  $\mathbf{a}$  into the corresponding diagonal matrix.  $\mathcal{N}(\mathbf{x}; \mathbf{a}, \mathbf{A})$  denotes the probability density function (PDF) of a Gaussian random vector  $\mathbf{x}$  with mean  $\mathbf{a}$  and covariance matrix  $\mathbf{A}$ , and the PDF of a complex-valued Gaussian random vector is denoted by  $\mathcal{N}_c(\cdot; \cdot, \cdot)$ .  $\mathbf{X}_{[:, \mathcal{A}]}$  ( $\mathbf{X}_{[\mathcal{A}, :]}$ ) refers to the sub-matrix comprising the columns (rows) of  $\mathbf{X}$  identified by the index set  $\mathcal{A}$ , while  $\otimes$  and  $\odot$  denotes Kronecker and Hadamard product operations, respectively. The operator  $\text{vec}(\cdot)$  stacks a matrix column by column into a vector, and  $[\mathbf{X}]_{i,j}$  is the  $(i, j)$ -th element of  $\mathbf{X}$ .  $\lfloor \cdot \rfloor$  denotes the integer floor operator.

## II. SYSTEM MODEL

In this study, we examine a typical narrowband uplink random access scenario where  $K$  potential UEs are served by a single BS. The BS is equipped with a uniform planar array (UPA) of  $M = M_x \times M_y$  antennas, with  $M_x$  and  $M_y$  antennas positioned along the  $x$ -axis and  $y$ -axis, respectively. For simplicity, each UE is assumed to use a single antenna. In a specific transmission frame, only  $K_a \ll K$  UEs are active due to the sporadic transmission characteristic of massive random access. The activity of a UE is represented by the indicator  $\alpha_k$ , where  $\alpha_k = 1$  indicates that the  $k$ -th UE is active, and  $\alpha_k = 0$  otherwise. By defining the AUS as  $\mathcal{A} = \{k \mid \alpha_k = 1, 1 \leq k \leq K\}$ , we have  $K_a = |\mathcal{A}|_c$ .

For the  $k$ -th active UE, the uplink frame consists of  $L_p$  pilot symbols  $\mathbf{p}_k \in \mathbb{C}^{L_p}$  followed by  $L_d$  data symbols  $\mathbf{x}_k \in \mathbb{C}^{L_d}$ . These pilot and data symbols form a frame  $\mathbf{s}_k = [\mathbf{p}_k^T, \mathbf{x}_k^T]^T \in \mathbb{C}^L$  of length  $L = L_p + L_d$ . The channel vector between the  $k$ -th UE and the BS is denoted by  $\mathbf{h}_k \in \mathbb{C}^M$ , which is assumed to remain constant throughout the entire transmission frame. The received signal matrix  $\mathbf{Y} \in \mathbb{C}^{L \times M}$  over a frame from all the

active UEs is expressed as:

$$\mathbf{Y} = \sum_{k=1}^K \alpha_k \mathbf{s}_k \mathbf{h}_k^T + \mathbf{W}, \quad (1)$$

where  $\mathbf{W} \in \mathbb{C}^{L \times M}$  is the noise matrix, whose elements are symmetric complex additive Gaussian white noise (AWGN) with zero mean and variance  $\sigma_n^2$ .

Define  $\mathbf{S} \triangleq [\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_K] \in \mathbb{C}^{L \times K}$ ,  $\mathbf{A} \triangleq \text{diag}(\boldsymbol{\alpha}) = \text{diag}([\alpha_1, \alpha_2, \dots, \alpha_K]^T) \in \mathbb{C}^{K \times K}$ ,  $\bar{\mathbf{H}} \triangleq [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_K]^T \in \mathbb{C}^{K \times M}$ , and  $\mathbf{H} \triangleq \mathbf{A} \bar{\mathbf{H}}$ . Then, the received signal matrix can be expressed as:

$$\mathbf{Y} = \mathbf{S} \mathbf{A} \bar{\mathbf{H}} + \mathbf{W} = \mathbf{S} \mathbf{H} + \mathbf{W}. \quad (2)$$

By splitting the pilot and data components in  $\mathbf{S}$  into two parts, i.e.,  $\mathbf{P} \triangleq [\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_K] \in \mathbb{C}^{L_p \times K}$  and  $\mathbf{X} \triangleq [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_K] \in \mathbb{C}^{L_d \times K}$ , we have  $\mathbf{S} = [\mathbf{P}^T, \mathbf{X}^T]^T \in \mathbb{C}^{L \times K}$  and  $\mathbf{Y} = [\mathbf{Y}_p^T, \mathbf{Y}_d^T]^T \in \mathbb{C}^{L \times M}$ , where  $\mathbf{Y}_p \in \mathbb{C}^{L_p \times M}$  and  $\mathbf{Y}_d \in \mathbb{C}^{L_d \times M}$  correspond to the received pilot and data components, respectively. We also denote  $\mathbf{W} = [\mathbf{W}_p^T, \mathbf{W}_d^T]^T \in \mathbb{C}^{L \times M}$  with  $\mathbf{W}_p \in \mathbb{C}^{L_p \times M}$  and  $\mathbf{W}_d \in \mathbb{C}^{L_d \times M}$ .

## III. PROBLEM FORMULATION

The proposed massive random access system is depicted in Fig. 1, where the BS needs to solve the problems of AUD and JCEDD. Specifically, we divide the AUD, CE, and DD tasks into the two stages. In the first stage, we use a single model to derive the AUS information from the received pilots of variable lengths. In the second stage, we perform the JCEDD using the powerful generation capability of diffusion models.

### A. Stage 1-AUD

Due to activity sparsity, existing CS methods [8]–[12] recover the entire sparse channel matrix, which requires a large pilot overhead to accurately estimate the CSI. As discussed in Section I, covariance-based methods [3]–[5] have not been successfully generalized to non-Rayleigh channels. Additionally, current deep learning-based methods [6], [7] fail to generalize to variable pilot lengths once the network is trained with a fixed pilot length. To address these issues, we propose a neural network-based method that can generalize to variable pilot

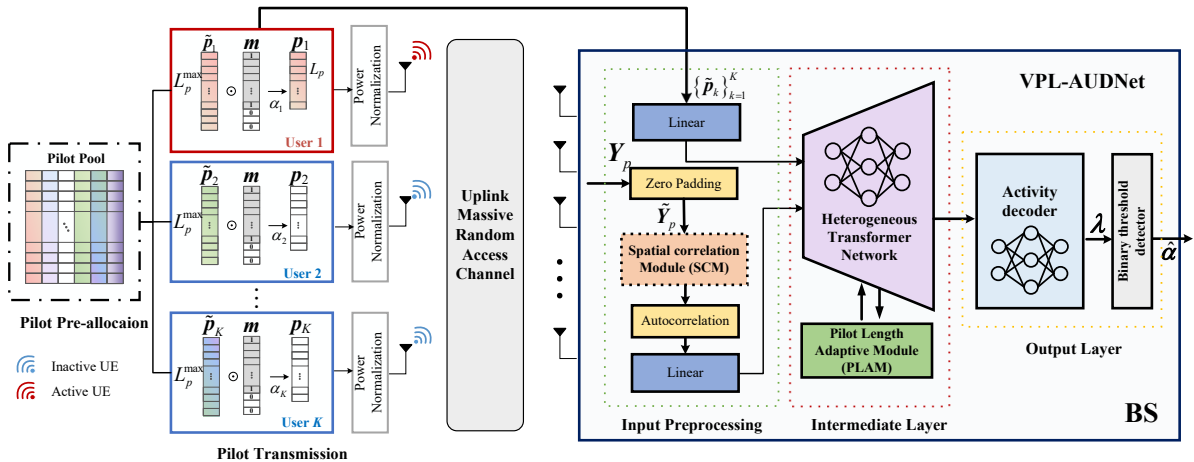


Fig. 1: System diagram of the massive random access with variable-length pilot transmission and corresponding VPL-AUDNet at the receiver.

lengths and perform well with low pilot overhead in complex correlated channel environments.

We assume that the original pilot sequence for each UE, denoted as  $\tilde{\mathbf{p}}_k$ , is selected from a large pilot pool  $\tilde{\mathbf{P}} = [\tilde{\mathbf{p}}_1, \tilde{\mathbf{p}}_2, \dots, \tilde{\mathbf{p}}_K] \in \mathbb{C}^{L_p^{\max} \times K}$  and is pre-allocated by the BS when the UE initially registers with the BS. The UE identity is then bound to this pilot sequence until it un-registers from the network. During the transmission procedure as shown in the left part of Fig. 1, only the first  $L_p$  symbols of each pilot sequence are transmitted. That is, each pilot sequence is masked by a binary vector  $\mathbf{m} = [1, \dots, 1, 0, \dots, 0]^T \in \{0, 1\}^{L_p^{\max}}$ , whose first  $L_p$  elements are ones and the remaining  $(L_p^{\max} - L_p)$  elements are zeros. Here,  $L_p$  can take a value in the range  $[L_p^{\min}, L_p^{\max}]$ . Let  $\mathbf{M} = \mathbf{1}_K^T \otimes \mathbf{m} \in \mathbb{B}^{L_p^{\max} \times K}$ . Then, at the AUD stage, the BS identifies the active UEs from the received pilot signals, expressed as:

$$\tilde{\mathbf{Y}}_p = (\tilde{\mathbf{P}} \odot \mathbf{M}) \mathbf{H} + \tilde{\mathbf{W}}_p. \quad (3)$$

Here,  $\tilde{\mathbf{Y}}_p = [\mathbf{Y}_p^T, \mathbf{0}_{(L_p^{\max} - L_p) \times M}]^T \in \mathbb{C}^{L_p^{\max} \times M}$  is the equivalent received pilot after zero-padding it to the maximum pilot sequence length. Similarly,  $\tilde{\mathbf{W}}_p = [\mathbf{W}_p^T, \mathbf{0}_{(L_p^{\max} - L_p) \times M}]^T \in \mathbb{C}^{L_p^{\max} \times M}$  is the equivalent noise at the receiver. Since the equivalent channel matrix can be written as  $\mathbf{H} = [\alpha_1 \mathbf{h}_1, \alpha_2 \mathbf{h}_2, \dots, \alpha_K \mathbf{h}_K]^T \in \mathbb{C}^{K \times M}$ , estimating the activity  $\hat{\alpha} = [\hat{\alpha}_1, \hat{\alpha}_2, \dots, \hat{\alpha}_K]^T \in \mathbb{B}^{K \times 1}$  is equivalent to recognizing the support set of  $\mathbf{H}$ .

The AUD problem can then be formulated as follows:

$$\begin{aligned} \min_{\Phi} \quad & \mathbb{E} \{ \mathcal{L}(\alpha, \hat{\alpha}) \}, \\ \text{s.t.} \quad & \hat{\alpha} = f_{\Phi}(\tilde{\mathbf{Y}}_p, \tilde{\mathbf{P}}, L_p), \forall L_p \in [L_p^{\min}, L_p^{\max}], \end{aligned} \quad (4)$$

where  $f_{\Phi}(\tilde{\mathbf{Y}}_p, \tilde{\mathbf{P}}, L_p)$  is a neural network which maps the received signal and pilot sequences to  $\hat{\alpha}$ ,  $\Phi$  represents the parameters of the neural network, and  $\mathcal{L}(\alpha, \hat{\alpha})$  is the loss function that quantifies the disparity between  $\alpha$  and  $\hat{\alpha}$ .

The advantage of variable pilot-length transmission is not only to avoid training and storing multiple neural networks for different pilot lengths but also to allow the UE to transmit different pilot lengths according to varying traffic densities at different times of the day. This reduces transmission delay while maintaining flexibility. The proposed AUD scheme is detailed Section IV.

### B. Stage 2-JCEDD

Given the estimated AUS  $\hat{\mathcal{A}} = \{k \mid \hat{\alpha}_k = 1, 1 \leq k \leq K\}$ , the transmission model can be equivalently formulated as:

$$\mathbf{Y} = \mathbf{S}_{[:, \hat{\mathcal{A}}]} \mathbf{H}_{[\hat{\mathcal{A}}, :]} + \tilde{\mathbf{W}}, \quad (5)$$

where  $\mathbf{S}_{[:, \hat{\mathcal{A}}]} = [\mathbf{P}_{[:, \hat{\mathcal{A}}]}^T, \mathbf{X}_{[:, \hat{\mathcal{A}}]}^T]^T \in \mathcal{X}^{L \times \hat{K}_a}$ , with  $\mathcal{X}$  denoting the quadrature amplitude modulation (QAM) constellation set, and  $\mathbf{H}_{[\hat{\mathcal{A}}, :]} \in \mathbb{C}^{\hat{K}_a \times M}$ . For convenience, define  $\mathbf{S}_a \triangleq \mathbf{S}_{[:, \hat{\mathcal{A}}]}$ ,  $\mathbf{P}_a \triangleq \mathbf{P}_{[:, \hat{\mathcal{A}}]}$ ,  $\mathbf{X}_a \triangleq \mathbf{X}_{[:, \hat{\mathcal{A}}]}$ , and  $\mathbf{H}_a \triangleq \mathbf{H}_{[\hat{\mathcal{A}}, :]}$ . Given the known observations  $\mathbf{Y}$  and pilots  $\mathbf{P}_a$ , we find the maximum a posteriori

estimation of both the data symbols  $\mathbf{X}_a$  and channels  $\mathbf{H}_a$ , which can be formulated as:

$$\{\hat{\mathbf{X}}_a, \hat{\mathbf{H}}_a\} = \arg \max_{\mathbf{X}_a \in \mathcal{X}^{L_d \times \hat{K}_a}, \mathbf{H}_a} p(\mathbf{X}_a, \mathbf{H}_a | \mathbf{Y}, \mathbf{P}_a). \quad (6)$$

The above problem can be divided into the two sub-problems when one of the two variables is assumed to be known:

$$\hat{\mathbf{X}}_a = \arg \max_{\mathbf{X}_a \in \mathcal{X}^{L_d \times \hat{K}_a}} p(\mathbf{X}_a | \mathbf{Y}, \hat{\mathbf{H}}_a, \mathbf{P}_a), \quad (7a)$$

$$\hat{\mathbf{H}}_a = \arg \max_{\mathbf{H}_a} p(\mathbf{H}_a | \mathbf{Y}, \hat{\mathbf{X}}_a, \mathbf{P}_a). \quad (7b)$$

Different from typical passing algorithms that factorize the joint distribution (6) into a factor graph and then inferring the posterior distribution of each element through message passing, our proposed scheme directly samples the signals from the posterior distributions (7a) and (7b). This approach was first implemented in [21], [25]–[27], where the ALD sampler is used to sample from the posterior distribution in the CE or DD problem.

The ALD sampler provides a viable method for sampling from complex distributions. Recent advances highlight the significant role of the score function in the ALD sampler, revealing a strong connection to the denoiser in the widely recognized DDPM model. Both concepts can be clarified through the perspective of SDE [32]. Specifically, sampling from a distribution can be interpreted as solving the reverse SDE of a corresponding forward SDE in the diffusion process. Based on this insight, we address the shortcomings of the existing ALD sampler [27] to provide more reliable estimation results for the JCEDD problem. The proposed JCEDD scheme is detailed in Section V.

### IV. TRANSFORMER-BASED AUD FOR VARIABLE PILOT LENGTHS

As illustrated in the right part of Fig. 1, the receiver's primary function is to extract correlations between the received signal and the user pilot sequences, facilitating the identification of active users whose pilots contribute to the received signal. We propose a VPL-AUDNet consisting of three main blocks. First, in the preprocessing block, we design a unified method for preprocessing received pilot signals with varying lengths. In the second block, the feature sequences from the previous block are concurrently fed into a neural network for correlation extraction, utilizing a heterogeneous Transformer network as the backbone [6] to handle the different physical meanings of the received signal and pilot sequences. To accommodate variable pilot lengths, we incorporate an additional pilot length adaptive module (PLAM) to make the Transformer network cognizant of key information from the input sequence. In the output layer, the activity decoder calculates the correlation between the feature sequences of  $\tilde{\mathbf{Y}}_p$  and the multi-user pilot sequences  $\{\tilde{\mathbf{p}}_k\}_{k=1}^K$  to estimate the user activity vector  $\lambda$ . The detailed AUD signal processing flow at the receiver is presented below.

#### A. Preprocessing

Before inputting the user pilots and received signals into the Transformer-network block, preprocessing is required to ensure dimensional consistency. The user pilots need to be converted

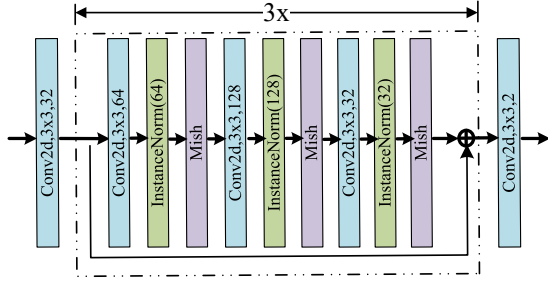


Fig. 2: Block diagram of the proposed spatial correlation module.

into real-valued representations, resulting in  $K$  sequences, denoted as  $\mathbf{z}_k^o \triangleq [\Re\{\tilde{\mathbf{p}}_k\}^T, \Im\{\tilde{\mathbf{p}}_k\}^T]^T \in \mathbb{R}^{2L_p^{\max}}$ ,  $1 \leq k \leq K$ , each of length  $2L_p^{\max}$ . These sequences are then processed through a linear layer to produce the pilot feature sequences  $\{\mathbf{z}_k^1\}_{k=1}^K$ , each of length  $d$ . Concurrently, to handle varying pilot transmission lengths  $L_p$  at the receiver, zero-padding is applied to the received signal  $\mathbf{Y}_p \in \mathbb{C}^{L_p \times M}$  to convert it into  $\tilde{\mathbf{Y}}_p \in \mathbb{C}^{L_p^{\max} \times M}$  (see (3)). Subsequently, to leverage the multi-antenna observations at the BS, the covariance matrix [4], [6] of the received signals is calculated as  $\mathbf{C} \triangleq \tilde{\mathbf{Y}}_p \tilde{\mathbf{Y}}_p^H / M \in \mathbb{C}^{L_p^{\max} \times L_p^{\max}}$ . This covariance matrix is transformed into a real-valued vectorized form, denoted as  $\mathbf{z}_{K+1}^o \triangleq [\Re\{\text{vec}(\mathbf{C})\}^T, \Im\{\text{vec}(\mathbf{C})\}^T]^T \in \mathbb{R}^{2(L_p^{\max})^2}$ . The resulting vector undergoes a linear layer transformation to achieve a dimensionality of  $d$ , denoted as  $\mathbf{z}_{K+1}^1$ . Finally, these  $K$  pilot sequences and the received signal sequence,  $\{\mathbf{z}_k^1\}_{k=1}^{K+1}$ , are collectively fed into the subsequent heterogeneous Transformer for further feature extraction.

In scenarios involving spatially correlated channels, directly mapping the covariance matrix onto a sequence may fail to effectively capture the antenna correlations. To address this limitation, we introduce a spatial correlation module (SCM) between the zero-padding and autocorrelation processes. Specifically, the received signal  $\tilde{\mathbf{Y}}_p \in \mathbb{C}^{L_{\max} \times M}$  is first transformed into a real tensor with dimensions  $L_{\max} \times 2 \times M_x \times M_y$  and then inputted into the SCM. To prevent the network parameters from being dependent on the number of antennas, the SCM uses a fully convolutional residual network to extract features along the vertical and horizontal antenna dimensions, as depicted in Fig. 2. The output of the SCM is reshaped to dimensions  $L_{\max} \times 2 \times M$  for subsequent autocorrelation calculation. For Rayleigh channels with independent identical distribution (i.i.d.), this module can be omitted. Simulation results in Section VI demonstrate that incorporating the SCM significantly enhances the performance of AUD in correlated channels such as 3GPP channels [33].

### B. Heterogeneous Transformer and Activity Decoder

Unlike the traditional Transformer, the heterogeneous Transformer introduced in [6] employs distinct sets of weight matrices for input embedding, multi-head attention (MHA), and the multi-layer perceptron to account for the different physical meanings of pilot sequences and received signals. Since the relative positional order of different sequences is not meaningful in our AUD task, we do not apply positional encoding as in conventional Transformers.

Assuming that the heterogeneous Transformer network consists of  $N$  encoder layers, we conduct activity estimation for  $K$  users based on the  $K+1$  sequences output by the  $N$ -

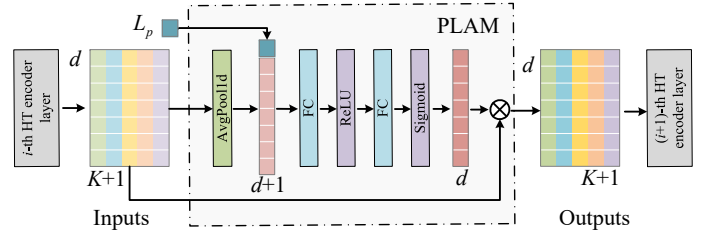


Fig. 3: Block diagram of the proposed pilot length adaptive module.

th encoder layer. This task is performed by the activity decoder, which computes the heterogeneous MHA between the final output feature sequence of the received signal  $\mathbf{z}_{K+1}^{N+1}$  and pilot sequences  $\{\mathbf{z}_k^{N+1}\}_{k=1}^K$ . The resulting MHA output,  $\mathbf{z}_c$ , represents the synthesized contributions of all pilot sequences to the received signal. Next, the user activity is estimated as the weighted correlation between  $\mathbf{z}_c$  and each pilot feature sequence given by:

$$\lambda'_k = C \tanh\left(\frac{\mathbf{z}_c^T \mathbf{W}^o \mathbf{z}_k^{N+1}}{\sqrt{d}}\right), \lambda_k = \frac{1}{1 + e^{-\lambda'_k}}, \forall k. \quad (8)$$

Here,  $\{\lambda_k\}_{k=1}^K$  is the output of the activity decoder,  $\mathbf{W}^o \in \mathbb{C}^{d \times d}$  is a learnable parameter matrix, and  $C$  is a hyperparameter empirically set to  $C = 10$ . User activity is then determined by a binary threshold detector:  $\hat{\alpha}_k = \mathbb{I}(\lambda_k > 0.5)$ ,  $\forall k$ , where the indicator  $\mathbb{I}(x) = 1$  if  $x$  is true, and 0 otherwise.

### C. Pilot Length Adaptive Module

In variable pilot-length transmission, the input sequence at the receiver undergoes zero-padding, causing some elements to carry no useful information. As a result, the significance of different elements in the input sequence varies. Allowing the neural network to recognize this variability enables adaptive optimization of network parameters based on the corresponding pilot lengths.

Inspired by the attention feature module in [34], we propose a PLAM. By inserting a PLAM module between each pair of adjacent Transformer encoder layers, we provide additional pilot length information to the receiver. The structure of the proposed PLAM module is illustrated in Fig. 3. Its input consists of  $(K+1)$  sequences from the output of the previous Transformer encoder layer, and its output comprises  $(K+1)$  sequences of the same dimension, which serve as the input to the next layer. First, an average pooling operation is applied to the  $(K+1)$  sequences along the sequence length dimension, producing a sequence of length  $d$ . This  $d$ -dimensional sequence is then concatenated with the pilot length information  $L_p$  and processed through a two-layer fully connected network, generating a  $d$ -dimensional weight sequence. Finally, this weight sequence is used for position-wise dot multiplication with all input sequences, redistributing the weights of the input feature sequence and allowing the receiver to recognize the varying importance of different elements in the inputs.

### D. Loss Function

We use binary cross-entropy as the loss function. The final loss is computed as the average of the loss values across

different pilot lengths, given by:

$$\mathcal{L} = \frac{1}{|\mathcal{D}|K} \sum_{i=1}^{|\mathcal{D}|} \sum_{k=1}^K \left( \alpha_k^{(i)} \log \lambda_k^{(i)} + \left(1 - \alpha_k^{(i)}\right) \left(1 - \log \lambda_k^{(i)}\right) \right). \quad (9)$$

Here,  $\mathcal{D}$  denotes the dataset, which consists of training samples  $\{\mathbf{Y}_p^{(i)}, \tilde{\mathbf{P}}^{(i)}, \boldsymbol{\alpha}^{(i)}\}$  with different pilot lengths  $L_p$ , where  $L_p \in [L_p^{\min}, L_p^{\max}]$ .

## V. GENERATIVE DIFFUSION MODELS DRIVEN JCEDD USING PREDICTOR-CORRECTOR SAMPLER

Given the active UE set, we need to recover the channel and corresponding data symbols of the active UEs. Before introducing the proposed JCEDD schemes, we briefly review the relationship between existing generative models and their SDE descriptions. We then address the CE problem, detailing the calculation of posterior score functions within the predictor-corrector (PC) sampler framework. Subsequently, we extend the application of the PC sampler to the DD problem. Finally, we combine the CE and DD methods to propose an iterative algorithm for JCEDD and analyze its complexity.

### A. Preliminary: Diffusion Process and SDE

Existing diffusion-based generative models involve corrupting data with multiple noise scales to form a known prior distribution. These models then learn to transform this prior distribution back into the data distribution by gradually removing noise. SDE considers a continuum of distributions that evolve over time according to a diffusion process. The forward diffusion process  $\mathbf{x}(t)$ , indexed by the continuous time variable  $t \in (0, T]$ , progressively diffuses a data point  $\mathbf{x}_0 \in \mathbb{R}^d$  sampled from a dataset with an unknown distribution  $p_0(\mathbf{x}) = p_{\text{data}}(\mathbf{x})$  into random noise  $\mathbf{x}(T)$  with distribution  $p_T(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \mathbf{0}, \sigma_T^2 \mathbf{I})$ . For simplicity in notation, we define  $p_t(\mathbf{x}) \triangleq p(\mathbf{x}(t))$ . This process can be described by an SDE that has no trainable parameters [32], [35], as follows:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}, \quad (10)$$

where  $\mathbf{f}(\mathbf{x}, t) : \mathbb{R}^d \times \mathbb{R} \rightarrow \mathbb{R}^d$  is a vector function called the drift coefficient, and the scalar function  $g(t) : \mathbb{R} \rightarrow \mathbb{R}$  is the diffusion coefficient. Additionally,  $\mathbf{w}$  is a standard Wiener process, defined as  $d\mathbf{w} = \mathbf{z}(t)dt$ , where  $\mathbf{z}(t) \sim \mathcal{N}(\mathbf{z}(t); \mathbf{0}, \mathbf{I})$ .

In the reverse process, starting from a sample  $\mathbf{x}_T \sim p_T(\mathbf{x})$ , random noise is smoothly transformed into a data sample  $\mathbf{x}_0 \sim p_0(\mathbf{x})$ . This reverse process satisfies a time-reversed SDE, which can be derived from the forward SDE given the score function  $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$  for the marginal probability of  $\mathbf{x}$  as a function of time, i.e.,

$$d\mathbf{x} = (\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}))dt + g(t)d\bar{\mathbf{w}}. \quad (11)$$

Here,  $p_t(\mathbf{x})$  denotes the probability of  $\mathbf{x}$  at time  $t$ , and  $\bar{\mathbf{w}}$  denotes the Wiener process when time flows backward from  $T$  to 0. Existing generative models like SMLD [22] and DDPM [29] can be considered specific numerical discretization solvers of the SDE. For instance, in SMLD, the coefficients are  $\mathbf{f}(\mathbf{x}, t) = \mathbf{0}$

and  $g(t) = \sqrt{\frac{d(\sigma^2(t))}{dt}}$ , with  $\sigma(t)$  serving as a predefined noise scheduling function. For DDPM, the drift and diffusion coefficients are given by  $\mathbf{f}(\mathbf{x}, t) = -\frac{\sigma(t)\mathbf{x}}{2}$  and  $g(t) = \sqrt{\sigma(t)}$ , respectively. By applying the discrete form of both the forward and reverse SDEs, we can derive the corresponding iteration equations used in SMLD and DDPM.

### B. Channel Estimation

As discussed in [32], the generation process involves solving the reversed SDE given in Eq. (11), where the unconditional generation from this equation yields samples that adhere to the prior probability distribution  $p_0(\mathbf{x})$ . However, considering Eq. (5) in the JCEDD problem, since the BS receives signal  $\mathbf{Y}$ , we need to consider the conditional generation process where channel should be drawn from the posterior distribution  $p(\mathbf{H}_a | \mathbf{Y}, \hat{\mathbf{X}}_a, \mathbf{P}_a)$  to produce the CE result that align with the constraint  $\mathbf{Y} = \mathbf{S}_a \mathbf{H}_a + \mathbf{W}$ .

Specifically, we assume that the data symbols  $\hat{\mathbf{X}}_a$  are known and represent the initial posterior distribution as  $p_0(\mathbf{H}_a | \mathbf{Y}) \triangleq p(\mathbf{H}_a(0) | \mathbf{Y})$ , where  $\mathbf{H}_a(0) = \mathbf{H}_a$  represents the ground truth channel to be estimated. The conditional distribution of  $\mathbf{H}_a(t)$  at time  $t$  is expressed as  $p_t(\mathbf{H}_a | \mathbf{Y}) \triangleq p(\mathbf{H}_a(t) | \mathbf{Y})$ . According to [36], the conditional generation process involves sampling from  $p_0(\mathbf{H}_a | \mathbf{Y})$ , which is equivalent to solving a reverse SDE represented as follows:

$$d\mathbf{H}_a = (\mathbf{f}(\mathbf{H}_a, t) - g(t)^2 \nabla_{\mathbf{H}_a} \log p_t(\mathbf{H}_a | \mathbf{Y}))dt + g(t)d\bar{\mathbf{W}}. \quad (12)$$

Due to the equivalence between DDPM and SMLD during the inference process [37], the remainder of this paper will examine the SDE in the SMLD form. Discussions on DDPM are similar and are therefore omitted.

To solve the reversed SDE (12), it is essential to derive the posterior score function  $\nabla_{\mathbf{H}_a} \log p_t(\mathbf{H}_a | \mathbf{Y})$ . By applying Bayes' theorem, the posterior score function is given by:

$$\nabla_{\mathbf{H}_a} \log p_t(\mathbf{H}_a | \mathbf{Y}) = \nabla_{\mathbf{H}_a} \log p_t(\mathbf{Y} | \mathbf{H}_a) + \nabla_{\mathbf{H}_a} \log p_t(\mathbf{H}_a), \quad (13)$$

where the *likelihood score function*  $\nabla_{\mathbf{H}_a} \log p_t(\mathbf{Y} | \mathbf{H}_a)$  is determined by the signal transmission model (5), and the *prior score function*  $\nabla_{\mathbf{H}_a} \log p_t(\mathbf{H}_a)$  is determined by the prior distribution of the channel dataset.

1) *Calculation of Prior Score Function:* Denote  $\mathbf{h}_k \triangleq [\mathbf{H}_a]_{[k,:]}^T \in \mathbb{C}^M$ ,  $\forall k \in \mathcal{A}$ . Since the prior distributions of each active UE's channel are assumed to be independent of each other, we have  $\log p_t(\mathbf{H}_a) = \log \prod_{k=1}^{K_a} p_t(\mathbf{h}_k) = \sum_{k=1}^{K_a} \log p_t(\mathbf{h}_k)$ . Therefore, the prior score function can be calculated for each UE separately, i.e.,

$$\nabla_{\mathbf{H}_a} \log p_t(\mathbf{H}_a) = \left[ \nabla_{\mathbf{h}_1} \log p_t(\mathbf{h}_1), \nabla_{\mathbf{h}_2} \log p_t(\mathbf{h}_2), \dots, \nabla_{\mathbf{h}_{K_a}} \log p_t(\mathbf{h}_{K_a}) \right]^T. \quad (14)$$

Therefore, when all the UEs' channels are generated from the same dataset distribution  $p_{\text{data}}(\mathbf{h})$ , we only need to focus on the prior score function of a single UE.

For simplicity, omit the UE index  $k$  and denote  $\mathbf{p}_0(\mathbf{h}) \triangleq p_{\text{data}}(\mathbf{h})$ . Our objective is to derive the score function

$\nabla_{\mathbf{h}} \log p_t(\mathbf{h})$ , where  $p_t(\mathbf{h})$  is the data distribution diffused by noise at time  $t$ . For complex channel distributions, the prior score function lacks a closed-form expression. Fortunately, the study [38] has shown that using a neural network to learn the score function  $\nabla_{\mathbf{h}} \log p_t(\mathbf{h})$  is equivalent to denoising score matching (DSM). This can be achieved by training a time-dependent neural network  $\mathbf{s}_{\theta}(\mathbf{h}(t), t)$  using the following loss function:

$$\mathcal{L}_s = \mathbb{E}_t \left\{ \lambda(t) \mathbb{E} \left\{ \left\| \mathbf{s}_{\theta}(\mathbf{h}(t), t) - \nabla_{\mathbf{h}(t)} \log p(\mathbf{h}(t) | \mathbf{h}(0)) \right\|_2^2 \right\} \right\}, \quad (15)$$

where the inner expectation is taken over the joint distribution  $p(\mathbf{h}(t), \mathbf{h}(0))$  with  $\mathbf{h}(0)$  sampled from the dataset distribution  $p_0(\mathbf{h})$ , and the time  $t$  is uniformly sampled from  $(0, 1]$  with  $T=1$ , while  $\lambda(t) \propto 1/\mathbb{E}[\|\nabla_{\mathbf{h}(t)} \log p(\mathbf{h}(t) | \mathbf{h}(0))\|_2^2]$  is a positive weight function, and  $\theta$  collects the neural network parameters.

Specifically,  $p(\mathbf{h}(t) | \mathbf{h}(0))$  denotes the transition kernel from  $\mathbf{h}(0)$  to  $\mathbf{h}(t)$ , typically defined by a forward noise diffusion process given by Eq. (10). For a typical generation model in the SMLD form [32], with a known drift coefficient  $\mathbf{f}(\mathbf{h}, t) = \mathbf{0}$  and a diffusion coefficient  $g(t) = \sqrt{\frac{d(\sigma^2(t))}{dt}}$ , the transition probability function from the original data point  $\mathbf{h}(0)$  to  $\mathbf{h}(t)$  can be derived using *Ito's Lemma* [35]. This is given by  $p(\mathbf{h}(t) | \mathbf{h}(0)) = \mathcal{N}(\mathbf{h}(t); \mathbf{h}(0), (\sigma^2(t) - \sigma^2(0)) \mathbf{I}) \approx \mathcal{N}(\mathbf{h}(t); \mathbf{h}(0), \sigma^2(t) \mathbf{I})$ , where  $\sigma(t) = \sigma_{\min} \left( \frac{\sigma_{\max}}{\sigma_{\min}} \right)^t$  is a predefined noise scheduling function, and the approximation holds when the initial noise variance  $\sigma^2(0) = \sigma_{\min}^2 \approx 0$ . Consequently, we have  $\nabla_{\mathbf{h}(t)} \log p(\mathbf{h}(t) | \mathbf{h}(0)) = -\frac{\mathbf{h}(t) - \mathbf{h}(0)}{\sigma^2(t)}$ . The final loss function can thus be simplified as:

$$\mathcal{L}_s = \mathbb{E}_t \left\{ \lambda(t) \mathbb{E} \left( \left\| \mathbf{s}_{\theta}(\mathbf{h}(t), t) + \frac{\mathbf{h}(t) - \mathbf{h}(0)}{\sigma^2(t)} \right\|_2^2 \right) \right\}, \quad (16)$$

which is equivalent to training a denoising neural network that can recover the scaled noise superposed on the channel. With sufficient data and model capacity, DSM ensures that the optimal solution  $\mathbf{s}_{\theta^*}(\mathbf{h}(t), t)$  approximates  $\nabla_{\mathbf{h}} \log p_t(\mathbf{h})$  for almost all  $\mathbf{h}(t)$  and  $t$ .

Given the powerful expressive capability of the noise conditional score network (NCSN++) [32] in image generation tasks, and considering that the channel can be viewed as a special kind of image, we use NCSN++ to construct  $\mathbf{s}_{\theta^*}(\mathbf{h}(t), t)$ . To adapt the channel dimension to the input of NCSN++, the original  $M$ -dimensional channel vector sample is transformed into a real representation and reshaped into dimensions  $2 \times M_x \times M_y$ . NCSN++ then takes the channel of these dimensions and a time step index  $t$  as inputs. The network outputs the learned channel prior score at time  $t$ . It should be noted that during the prior score function learning, no information on the pilot symbols, data symbols, received signal, or transmission noise variance is involved in the training process. This means that once the neural network is trained, it can be applied to any dimension of the JCEDD problem as long as the channel distribution remains unchanged.

**2) Calculation of Likelihood Score Function:** We now consider how to derive  $\nabla_{\mathbf{H}_a} \log p_t(\mathbf{Y} | \mathbf{H}_a)$ . Given that the channels are viewed as three-dimensional real tensors in prior score learning, we can also transform the transmission model (5) into its corresponding real-valued representation.

For simplicity, we retain the notation  $\mathbf{Y} = \mathbf{S}_a \mathbf{H}_a + \mathbf{W}$  throughout this paper. Note that in the SMLD framework,  $p(\mathbf{H}_a(t) | \mathbf{H}_a(0)) \approx \mathcal{N}(\mathbf{H}_a(t); \mathbf{H}_a(0), \sigma^2(t) \mathbf{I})$ . Consequently, the transmission model can equivalently be expressed as:

$$\mathbf{Y} = \mathbf{S}_a (\mathbf{H}_a(t) + \sigma(t)^2 \mathbf{Z}) + \mathbf{W}, \quad (17)$$

where  $\mathbf{Z} \sim \mathcal{N}(\mathbf{Z}; \mathbf{0}, \mathbf{I})$  is the diffusion noise superimposed on the original channel  $\mathbf{H}_a(0)$ . Therefore, the corresponding transition probability is given by:

$$p_t(\mathbf{Y} | \mathbf{H}_a) \sim \mathcal{N}(\mathbf{Y}; \mathbf{S}_a \mathbf{H}_a(t), \sigma_n^2 \mathbf{I} + \sigma^2(t) \mathbf{S}_a \mathbf{S}_a^T). \quad (18)$$

Taking the gradient of the log-likelihood, we obtain:

$$\nabla_{\mathbf{H}_a} \log p_t(\mathbf{Y} | \mathbf{H}_a) = \mathbf{S}_a^T (\sigma_n^2 \mathbf{I} + \sigma^2(t) \mathbf{S}_a \mathbf{S}_a^T)^{-1} (\mathbf{Y} - \mathbf{S}_a \mathbf{H}_a(t)). \quad (19)$$

In the considered JCEDD problem,  $\mathbf{S}_a = [\mathbf{P}_a^T, \hat{\mathbf{X}}_a^T]^T$  contains both the pilot symbol  $\mathbf{P}_a$  and estimated data symbol  $\hat{\mathbf{X}}_a$  in the iteration process, and the estimated data symbol can enhance the CE. When no data symbol is used, the problem degrades to the conventional pure pilot-aided CE problem.

**3) Channel Sampling Process:** Utilizing the derived prior score function and likelihood score function, we numerically solve the reverse SDE (12). Specifically, by dividing the interval  $t \in (0, 1]$  into  $N$  discrete steps, indexed as  $i=0, 1, \dots, N-1$ , we obtain the following iterative rule for the reverse diffusion sampler (which is also called *predictor*):

$$\begin{aligned} \mathbf{H}_a^i &= \mathbf{H}_a^{i+1} - \mathbf{f}_{i+1}(\mathbf{H}_a^{i+1}) + g_{i+1}^2 \nabla_{\mathbf{H}_a^{i+1}} \log p(\mathbf{H}_a^{i+1} | \mathbf{Y}) \\ &\quad + g_{i+1} \mathbf{Z}, \end{aligned} \quad (20)$$

where  $\mathbf{H}_a^i = \mathbf{H}_a(t)|_{t=\frac{i}{N}}$ ,  $\mathbf{f}_i(\mathbf{H}_a^i) = \mathbf{f}(\mathbf{H}_a(t), t)|_{t=\frac{i}{N}}$ ,  $g_i = g(t)|_{t=\frac{i}{N}}$ , and  $\mathbf{Z} \sim \mathcal{N}(\mathbf{Z}; \mathbf{0}, \mathbf{I})$ . Given the known drift coefficient  $\mathbf{f}(\mathbf{H}_a, t) = \mathbf{0}$  and diffusion coefficient  $g(t) = \sqrt{\frac{d(\sigma^2(t))}{dt}}$  in SMLD, Eq. (20) is equivalent to:

$$\begin{aligned} \tilde{\mathbf{H}}_a^i &= \mathbf{H}_a^{i+1} + (\sigma_{i+1}^2 - \sigma_i^2) \nabla_{\mathbf{H}_a^{i+1}} \log p(\mathbf{H}_a^{i+1} | \mathbf{Y}) \\ &\quad + \sqrt{\sigma_{i+1}^2 - \sigma_i^2} \mathbf{Z}, \end{aligned} \quad (21)$$

where  $\sigma_i = \sigma(t)|_{t=\frac{i}{N}}$ . Then, starting from a random initialization  $\mathbf{H}_a^N \sim \mathcal{N}(\mathbf{H}_a^N; \mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ , the iteration process for the SMLD-based predictor is shown in Fig. 4. The corresponding discretized form of the score function is given as:

$$\nabla_{\mathbf{H}_a^i} \log p(\mathbf{H}_a^i) = [\mathbf{s}_{\theta^*}(\mathbf{h}_1^i, \frac{i}{N}), \dots, \mathbf{s}_{\theta^*}(\mathbf{h}_{K_a}^i, \frac{i}{N})]^T, \quad (22a)$$

$$\begin{aligned} \nabla_{\mathbf{H}_a^i} \log p(\mathbf{Y} | \mathbf{H}_a^i) &= \mathbf{V} \Sigma^T (\sigma_n^2 \mathbf{I} + \sigma_i^2 \Sigma \Sigma^T)^{-1} \\ &\quad \times (\mathbf{U}^T \mathbf{Y} - \Sigma \mathbf{V}^T \mathbf{H}_a^i), \end{aligned} \quad (22b)$$

$$\begin{aligned} \nabla_{\mathbf{H}_a^i} \log p(\mathbf{H}_a^i | \mathbf{Y}) &= \lambda_h \nabla_{\mathbf{H}_a^i} \log p(\mathbf{Y} | \mathbf{H}_a^i) \\ &\quad + \nabla_{\mathbf{H}_a^i} \log p(\mathbf{H}_a^i). \end{aligned} \quad (22c)$$

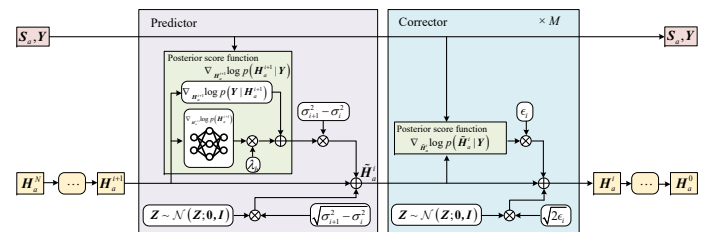


Fig. 4: Block diagram of the proposed PC sampler-based CE algorithm.

In Eq. (22b), we apply singular value decomposition (SVD) to  $S_a = U\Sigma V^T$  within Eq. (19) to avoid non-diagonal matrix inversions, thereby reducing computational complexity. Additionally, a scaling parameter  $\lambda_h = 2.5$ , as suggested in [39], is empirically chosen to balance the effects of the likelihood score and prior score, enhancing convergence performance.

However, the numerical predictor in Eq. (21) may introduce approximation errors when solving the continuous SDE in a discrete manner. To correct the errors, a score-based Markov Chain Monte Carlo (MCMC) method is used as the *corrector*. This method corrects the marginal distribution of the estimated sample during iterations. A well-known MCMC approach is the ALD, which employs the following iterative process:

$$\mathbf{H}_a^i = \tilde{\mathbf{H}}_a^i + \epsilon_i \nabla_{\tilde{\mathbf{H}}_a^i} \log p(\tilde{\mathbf{H}}_a^i | \mathbf{Y}) + \sqrt{2\epsilon_i} \mathbf{Z}, \quad (23)$$

where  $\epsilon_i = 2\sigma_i (r \|\mathbf{Z}\|_2 / \|\nabla_{\tilde{\mathbf{H}}_a^i} \log p(\tilde{\mathbf{H}}_a^i | \mathbf{Y})\|_2)^2$  is the step size in the  $i$ -th iteration of the ALD algorithm [32]. Additionally,  $r$  is an empirically chosen hyperparameter, set to  $r = 0.3$  in our simulations. Existing ALD-based studies on CE [21], DD [25], and JCEDD [27] can be regarded as corrector-only samplers with potential for performance improvement. Therefore, we address the JCEDD problem using the PC sampler illustrated in Fig. 4.

4) *Overall CE Algorithm*: In practical implementations, each iteration of the PC sampler includes one prediction step and  $Q$  correction steps, where  $Q$  ranges from 1 to 3. The complete CE algorithm is listed as Algorithm 1. Compared to traditional Bayesian algorithms, our method derives prior distribution knowledge through network learning, which avoids the mismatches associated with manually specified prior distributions. Moreover, unlike purely data-driven or model-driven deep learning algorithms based on supervised learning, this method does not require training separate networks for different numbers of active UEs and varying pilot lengths, offering improved generalization and interpretability.

### C. Data Detection

Upon reviewing the formulas (2) and (5), the DD problem can be characterized as  $\mathbf{Y}_d = \mathbf{X}_a \mathbf{H}_a + \mathbf{W}_d$ . By transposing the formula and representing the model in a real-valued form, the signal transmission model can be reformulated as:

$$\mathbf{\hat{Y}}_d = \mathbf{\hat{H}}_a \mathbf{\hat{X}}_a + \mathbf{\hat{W}}_d, \quad (24)$$

where  $\mathbf{\hat{Y}}_d$ ,  $\mathbf{\hat{H}}_a$ ,  $\mathbf{\hat{X}}_a$  and  $\mathbf{\hat{W}}_d$  are the transposed counterparts of the corresponding variables in real-valued form. Similar to the CE problem, assuming that the channel  $\mathbf{\hat{H}}_a$  is known, data symbols can be sampled from the posterior distribution  $p(\mathbf{\hat{X}}_a | \mathbf{\hat{Y}}_d)$  by solving the SDE, which also involves calculating the prior score and the likelihood score of the data. Since the data symbols are uniformly sampled from a QAM constellation, the prior score  $\nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{X}}_a^i)$  in the  $i$ -th iteration step can be calculated using Tweedie's formula [40], i.e.,

$$\nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{X}}_a^i) = \frac{\mathbb{E}\{\mathbf{\hat{X}}_a^0 | \mathbf{\hat{X}}_a^i\} - \mathbf{\hat{X}}_a^i}{\tau_i^2}, \quad (25)$$

where  $\tau_i = \tau(t)|_{t=\frac{i}{N}}$  is the scheduled perturbation noise for data symbol, and  $\tau(t) = \tau_{\min} \left( \frac{\tau_{\max}}{\tau_{\min}} \right)^t$  is a predefined scheduling

---

### Algorithm 1 PC Sampler-Based Channel Estimation

---

**Input:**  $S_a$ ,  $\mathbf{Y}$ ,  $\sigma_n^2$ , pre-trained denoising network  $\mathbf{s}_{\theta^*}$ , step number  $N$ , noise scheduling function  $\sigma(t)$ , and hyperparameter  $\lambda_h$ ;

- 1: **Initialize:**  $\mathbf{H}_a^N \sim \mathcal{N}(\mathbf{H}_a^N; \mathbf{0}, \sigma_N^2 \mathbf{I})$ ;
- 2: **Compute:**  $S_a = U\Sigma V^T$ ;
- 3: **for**  $i = N$  to 1 **do**
- 4:   Calculate  $\nabla_{\mathbf{H}_a^i} \log p(\mathbf{H}_a^i | \mathbf{Y})$  via (22);
- 5:   Draw  $\mathbf{Z} \sim \mathcal{N}(\mathbf{Z}; \mathbf{0}, \mathbf{I})$ ;
- 6:    $\mathbf{H}_a^{i-1} = \mathbf{H}_a^i + (\sigma_i^2 - \sigma_{i-1}^2) \nabla_{\mathbf{H}_a^i} \log p(\mathbf{H}_a^i | \mathbf{Y}) + \sqrt{\sigma_i^2 - \sigma_{i-1}^2} \mathbf{Z}$ ;
- 7:   **for**  $j = 1$  to  $Q$  **do**
- 8:     Draw  $\mathbf{Z} \sim \mathcal{N}(\mathbf{Z}; \mathbf{0}, \mathbf{I})$ ;
- 9:     Calculate  $\nabla_{\mathbf{H}_a^{i-1}} \log p(\mathbf{H}_a^{i-1} | \mathbf{Y})$  via (22);
- 10:      $\epsilon_{i-1} = 2\sigma_{i-1} \left( r \|\mathbf{Z}\|_2 / \|\nabla_{\mathbf{H}_a^{i-1}} \log p(\mathbf{H}_a^{i-1} | \mathbf{Y})\|_2 \right)^2$ ;
- 11:      $\mathbf{H}_a^{i-1} = \mathbf{H}_a^{i-1} + \epsilon_{i-1} \nabla_{\mathbf{H}_a^{i-1}} \log p(\mathbf{H}_a^{i-1} | \mathbf{Y}) + \sqrt{2\epsilon_{i-1}} \mathbf{Z}$ ;
- 12:   **end for**
- 13: **end for**

**Output:** Estimated channel  $\hat{\mathbf{H}}_a = \mathbf{H}_a^0$ .

---

function. Additionally, the posterior distribution can be derived using Bayes' rule as:  $p(\mathbf{\hat{X}}_a^i | \mathbf{\hat{X}}_a^i) = \frac{p(\mathbf{\hat{X}}_a^i | \mathbf{\hat{X}}_a^0) p(\mathbf{\hat{X}}_a^0)}{\sum_{\mathbf{\hat{X}}_a^0 \sim \mathcal{X}} p(\mathbf{\hat{X}}_a^i | \mathbf{\hat{X}}_a^0) p(\mathbf{\hat{X}}_a^0)}$ , where  $p(\mathbf{\hat{X}}_a^0) = \frac{1}{|\mathcal{X}|}$ . Then, the posterior mean is derived as

$$\mathbb{E}\{\mathbf{\hat{X}}_a^0 | \mathbf{\hat{X}}_a^i\} = \frac{\sum_{\mathbf{\hat{X}}_a^0 \sim \mathcal{X}} \mathbf{\hat{X}}_a^0 p(\mathbf{\hat{X}}_a^i | \mathbf{\hat{X}}_a^0)}{\sum_{\mathbf{\hat{X}}_a^0 \sim \mathcal{X}} p(\mathbf{\hat{X}}_a^i | \mathbf{\hat{X}}_a^0)}. \quad (26)$$

By substituting (26) into (25), the final prior score the data symbols at time step  $i$  can be expressed as

$$\nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{X}}_a^i) = \frac{1}{\tau_i^2} \left( \frac{\sum_{\mathbf{\hat{X}}_a^0 \sim \mathcal{X}} \mathbf{\hat{X}}_a^0 e^{-\frac{\|\mathbf{\hat{X}}_a^i - \mathbf{\hat{X}}_a^0\|^2}{2\tau_i^2}}}{\sum_{\mathbf{\hat{X}}_a^0 \sim \mathcal{X}} e^{-\frac{\|\mathbf{\hat{X}}_a^i - \mathbf{\hat{X}}_a^0\|^2}{2\tau_i^2}}} - \mathbf{\hat{X}}_a^i \right). \quad (27)$$

For the likelihood score, the equivalent likelihood function can be derived from the transmission model (24) as:

$$\nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{Y}}_d | \mathbf{\hat{X}}_a^i) = \mathbf{V}_h \Sigma_h^T (\sigma_n^2 \mathbf{I}_{2M} + \tau_i^2 \Sigma_h \Sigma_h^T)^{-1} \times (\mathbf{U}_h^T \mathbf{\hat{Y}}_d - \Sigma_h \mathbf{V}_h^T \mathbf{\hat{X}}_a^i), \quad (28)$$

where we have used the SVD  $\mathbf{\hat{H}}_a = \mathbf{U}_h \Sigma_h \mathbf{V}_h^T$  to reduce the complexity during the iteration. Finally, the posterior score of the data symbols can be calculated as

$$\nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{X}}_a^i | \mathbf{\hat{Y}}_d) = \lambda_x \nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{Y}}_d | \mathbf{\hat{X}}_a^i) + \nabla_{\mathbf{\hat{X}}_a^i} \log p(\mathbf{\hat{X}}_a^i), \quad (29)$$

where  $\lambda_x = 2.5$  is an empirically chosen hyperparameter. The DD algorithm can also be derived based on PC-sampler. Due to space constraint, the detailed DD algorithm is omitted, but its basic iterative procedure is similar to that of Algorithm 1.



score's complexity is  $\mathcal{O}(M^2 L_d + K_a^2 L_d)$ , and the data prior score's complexity is  $\mathcal{O}(2K_a L_d |\mathcal{X}|_c)$ .

For typical simulation parameters, e.g.,  $N_{\text{update}} = 50$ ,  $\lfloor \frac{i^*}{N_{\text{update}}} \rfloor = 5 \sim 15$ ,  $Q = 3$ ,  $N_X = 1500$  and  $N_H = 1500$ , the algorithm's overall complexity is mainly determined by the score function computations in Lines 4-5 and Lines 10-11. In subsequent simulations, the computational delay of the proposed algorithm will be compared with those of other baseline algorithms to verify its complexity efficacy.

## VI. NUMERICAL RESULTS

In this section, we first detail the experimental setup, including simulation parameters, network configurations, channel datasets, training conditions, and comparative algorithms. We then assess the performance of the Transformer-based AUD as well as the generative diffusion models driven JCEDD.

### A. Experimental Setups

We consider a typical uplink random access scenario with a total of  $K = 128$  potential users, where each user's activity follows a Bernoulli distribution with an active probability of 0.1. The BS employs a UPA with  $M_x = M_y = 8$  antennas on each dimension, while each user operates with a single antenna. Each element of the pilot sequence  $\tilde{p}_k, \forall k$ , and the data symbols  $x_k$  are randomly sampled from 4QAM constellations. The maximum pilot sequence length is  $L_p^{\max} = 28$ . We consider a 3GPP-compliant channel model [33] generated using Quadriga [41], simulating the 3GPP 38.901 UMa NLOS scenario at a carrier frequency of 3.5 GHz. The minimum pilot sequence length is  $L_p^{\min} = 8$  for Rayleigh channel, and  $L_p^{\min} = 10$  for 3GPP channel, since 3GPP channel is more complicated and it needs more pilots to estimate AUS.

1) *VPL-AUDNet Setup*: A total of 64,000 channel samples are generated using Quadriga. For each training batch, the pilot lengths for the UEs are randomly selected from  $[L_p^{\min}, L_p^{\max}]$ . In addition to the input preprocessing layer and output layer, the VPL-AUDNet contains 5 heterogeneous Transformer encoder layers, each featuring a hidden state sequence length of  $d=512$  and 8 attention heads. Training is conducted over 200 epochs, with the learning rate warming up to  $10^{-4}$  during the initial 15 epochs and then decaying to 0.1 of this original value every 75 epochs. The batch size is set to 128.

2) *Channel Prior Score Network Setup*: The NSCN++ architecture [32] is employed to model the prior score function  $s_{\theta}(\mathbf{h}(t), t)$ . A 3GPP training dataset consisting of 160,000 channel samples is utilized. The time step  $t$  is uniformly sampled from the interval  $[0, 1]$ , and the perturbation noise variance is determined using  $\sigma(t) = \sigma_{\min} \left( \frac{\sigma_{\max}}{\sigma_{\min}} \right)^t$ , with  $\sigma_{\max} = 30$  and  $\sigma_{\min} = 0.01$ . The network is trained with a learning rate of  $10^{-5}$  and a batch size of 64, over 20 epochs.

During the testing phase, the reverse SDE process for CE is discretized into  $N_H = 1,500$  steps. Given that an LMMSE initialization is employed, only  $i^*$  of the  $N_H$  steps are executed. The reverse SDE process for DD is also discretized into  $N_X = 1,500$  steps, using a noise scheduling function defined as  $\tau(t) = \tau_{\min} \left( \frac{\tau_{\max}}{\tau_{\min}} \right)^t$ , where  $\tau_{\max} = 1$  and  $\tau_{\min} = 0.01$ . Additionally, the hyperparameters are set to  $\lambda_h = \lambda_x = 2.5$ .

3) *Evaluation Metrics*: We utilize activity error probability (AEP), normalized mean square error (NMSE) and bit error rate (BER) to evaluate the performance of AUD, CE and DD, respectively. These metrics are defined respectively as  $\text{AEP} = \mathbb{E} \left\{ \frac{1}{K} \sum_{k=1}^K |\alpha_k - \hat{\alpha}_k| \right\}$ ,  $\text{NMSE} = \mathbb{E} \left\{ \frac{\|\mathbf{H}_a - \hat{\mathbf{H}}_a\|^2}{\|\mathbf{H}_a\|^2} \right\}$  and  $\text{BER} = \mathbb{E} \left\{ \frac{E_b}{K_a L_d \log_2 |\mathcal{X}|_c} \right\}$ , where  $E_b$  denotes the total number of error bits for active UEs.

4) *Comparative Schemes*: For AUD, the proposed method is compared with two baseline algorithms: AMP [9] and OMP [8]. Additionally, we also assess the performance of the VPL-AUDNet scheme trained with a fixed pilot length. For CE and DD, the following algorithms are included in the comparisons:

- **LS-CE & Perfect-Data**: The data  $\mathbf{X}_a$  is perfectly known, and jointly utilize pilots  $\mathbf{P}_a$  and data  $\mathbf{X}_a$  for LS CE;
- **Perfect-CSI & ZF-DD**: The CSI  $\mathbf{H}_a$  is perfectly known, and employ ZF for DD;
- **Perfect-CSI & OAMP-DD**: The CSI  $\mathbf{H}_a$  is perfectly known, and utilize OAMP [16] for DD;
- **Pilot LS-CE & ZF-DD**: Utilize pilots to perform LS CE, and use the estimated channel for ZF DD;
- **Pilot LS-CE & OAMP-DD**: Utilize pilots to perform LS CE, and use the estimated channel for OAMP [16] DD;
- **Iter LMMSE-CE & OAMP-DD**: Employ LMMSE for CE and OAMP for DD, with CE and DD iteratively executed to enhance the performance of each other [16];
- **Iter Langevin CE & DD**: Conduct iterative CE and DD using ALD [27];
- **SDE-CE & Perfect-Data**: The data  $\mathbf{X}_a$  is perfectly known, and apply SDE solver (i.e., PC sampler) for CE, which corresponds to Algorithm 1;
- **Perfect-CSI & SDE-DD**: The CSI  $\mathbf{H}_a$  is known, and apply SDE solver for DD;
- **Our Iter SDE CE & DD**: The proposed Algorithm 2.

### B. Performance of AUD

Fig. 6(a) illustrates the AUD performance of various schemes as the functions of the transmission pilot length,  $L_p^{\text{test}}$ , under i.i.d. Rayleigh channels, where the channel between each potential UE and the BS is modeled as  $\mathbf{h}_k \sim \mathcal{N}_c(\mathbf{h}_k; \mathbf{0}, \mathbf{I}_M), \forall k$ . The simulation is conducted under the received SNR  $= \frac{\mathbb{E}\{\|\mathbf{S}\mathbf{H}\|^2\}}{\mathbb{E}\{\|\mathbf{W}\|^2\}} = 10$  dB. The results indicate that the VPL-AUDNet-based schemes outperform traditional CS algorithms based on AMP and OMP, in terms of AEP. Furthermore, the VPL-AUDNet models trained with fixed pilot lengths and without the PLAM module, e.g., VPL-AUDNet (w/o PLAM),  $L_p^{\text{train}} = 15$ , exhibit degraded performance when tested on pilot lengths different

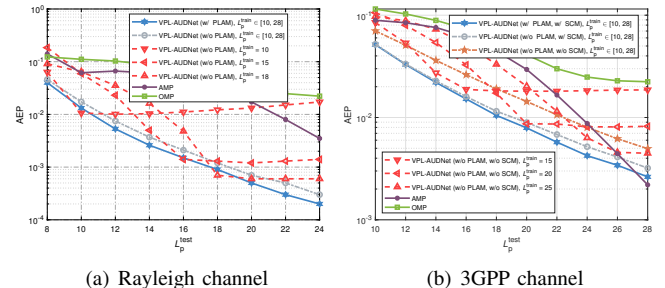


Fig. 6: AEP performance with respect to the transmission pilot length of different AUD schemes.

from the training pilot length. Conversely, VPL-AUDNet (w/o PLAM),  $L_p^{\text{train}} \in [10, 28]$  exhibit better generalization performance. Moreover, the proposed VPL-AUDNet (w/ PLAM),  $L_{\text{train}} \in [8, 24]$ , outperforms VPL-AUDNet (w/o PLAM),  $L_{\text{train}} \in [8, 24]$ , demonstrating the effectiveness of the PLAM. Additionally, under i.i.d. Rayleigh channels, the proposed VPL-AUDNet scheme achieves an AEP performance close to  $10^{-2}$  even when the pilot number is less than the average number of UEs ( $L_p^{\text{test}} \leq 12$ ), whereas the AMP and OMP algorithms exhibit a AEP of  $10^{-1}$ . These results demonstrate that the proposed scheme achieves optimal overall performance across various pilot lengths within a single model, thereby confirming its effectiveness.

Fig. 6(b) compares the AUD performance of various schemes as the functions of the number of transmission pilots  $L_p^{\text{test}}$  under 3GPP channels, given a received SNR of 10 dB. The results show that the performance curves of the different schemes are similar to the trends observed under Rayleigh channels. Compared to traditional CS algorithms like AMP and OMP, the VPL-AUDNet-based scheme achieves better performance with significantly reduced pilot length by avoiding completely CE. Additionally, a comparison between the curves of VPL-AUDNet (w/o PLAM, w/ SCM),  $L_p^{\text{train}} = [10, 28]$ , and VPL-AUDNet (w/o PLAM, w/o SCM),  $L_p^{\text{train}} = [10, 28]$ , reveals that the SCM significantly enhances AEP performance under 3GPP channels. Moreover, VPL-AUDNet (w/ PLAM, w/ SCM),  $L_p^{\text{train}} = [10, 28]$  scheme attains optimal performance at low pilot numbers.

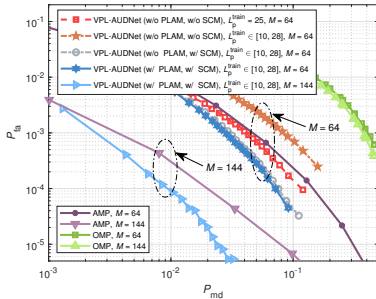


Fig. 7:  $P_{\text{fa}}-P_{\text{md}}$  performance of different AUD schemes under 3GPP channel.

Fig. 7 depicts the curves for the probability of false alarm ( $P_{\text{fa}}$ ) and the probability of missed detection ( $P_{\text{md}}$ ) for various schemes under 3GPP channels. Simulations are conducted with a received SNR of 10 dB and a test pilot length of  $L_p^{\text{test}} = 25$ . All VPL-AUDNet models are trained with an antenna number of  $M = 64$ . The results indicate that the proposed VPL-AUDNet (w/ PLAM, w/ SCM),  $L_p^{\text{train}} = [10, 28]$ , achieves an optimal trade-off between  $P_{\text{fa}}$  and  $P_{\text{md}}$ . Moreover, testing the model trained with  $M = 64$  antennas on the channels with  $M = 144$  antennas results in further improvements in  $P_{\text{fa}}-P_{\text{md}}$  performance, demonstrating the scheme's robust generalization capability across varying antenna configurations. In contrast, the performance gains for OMP related to an increased number of antennas are less significant.

### C. Performance of JCEDD

In the simulations, unless otherwise specified, the BS is equipped with  $M = 64$  antennas, and a total of  $K_a = 12$  active UEs access the system simultaneously.

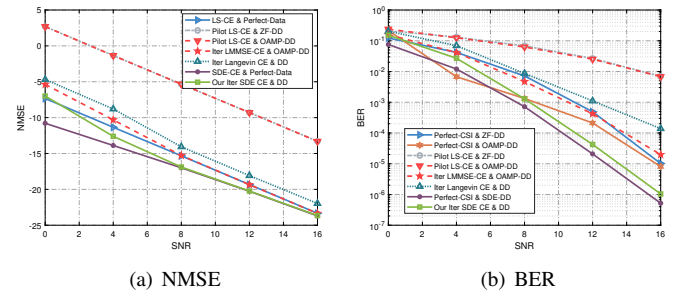


Fig. 8: NMSE and BER performance with respect to the received SNR for different CE and DD algorithms.

Fig. 8 illustrates the performance curves of CE and DD under varying SNRs for different schemes. In the simulations, each user is allocated a pilot length of  $L_p = 15$  and a data length of  $L_d = 50$ . As shown in Fig. 8(a), the iterative CE and DD schemes (i.e., **Iter Langevin CE & DD**, **Iter LMMSE-CE & OAMP-DD**, and **Our Iter SDE CE & DD**) achieve an NMSE gain of approximately 7 and 10 dB when compared to the non-iterative **Pilot LS-CE & ZF-DD**, and **Pilot LS-CE & OAMP-DD** schemes. This indicates a significant gain in CE performance by using detected data to refine CE. As SNR increases, **Iter LMMSE-CE & OAMP-DD**'s performance approaches that of **LS-CE & Perfect-Data**. Similarly, the performance of **Our Iter SDE CE & DD** (Algorithm 2) matches **SDE-CE & Perfect-Data** (Algorithm 1) for  $\text{SNR} \geq 8$  dB. When the data is not perfectly known, **Our Iter SDE CE & DD** outperforms **Iter LMMSE-CE & OAMP-DD** because it more effectively utilizes the prior distribution of the channel through a learned prior score function, whereas **Iter LMMSE-CE & OAMP-DD** relies only on the channel's second-order statistical information via LMMSE. Additionally, **Our Iter SDE CE & DD** implements a PC sampler to enhance SDE solution performance beyond what **Iter Langevin CE & DD** algorithm can achieve with only a corrector.

Fig. 8(b) compares the BER performance of various algorithms. Similar to Fig. 8(a), the non-iterative **Pilot LS-CE & ZF-DD** and **Pilot LS-CE & OAMP-DD** algorithms perform worse than the iterative algorithms. **Our Iter SDE CE & DD** scheme achieves performance close to that of **Perfect-CSI & SDE-DD** and surpasses the **Iter LMMSE-CE & OAMP-DD** and **Iter Langevin CE & DD** algorithms. At a BER of  $10^{-4}$ , **Our Iter SDE CE & DD** offers a 3 dB SNR gain over **Iter LMMSE-CE & OAMP-DD**. Furthermore, **Perfect-CSI & SDE-DD** outperforms **Perfect-CSI & OAMP-DD** and **Perfect-CSI & ZF-DD** algorithms. This is likely because the channel matrix elements for the 3GPP scenario do not meet the i.i.d. distribution assumptions for DD tasks, causing OAMP algorithms to underperform. On the other hand, the ZF algorithm does not exploit the prior distribution information of the constellation. In contrast, **Perfect-CSI & SDE-DD** thoroughly leverages the prior distribution of the data and the likelihood information of the received signals, resulting in superior performance.

Fig. 9 compares the performance curves of CE and DD as the functions of pilot length  $L_p$  for different schemes. In the simulation, each user's data length is  $L_d = 50$ , and the received SNR at the BS is set to 10 dB. As shown in Fig. 9(a), when  $L_p > 12$ , **Our Iter SDE CE & DD** can achieve the performance

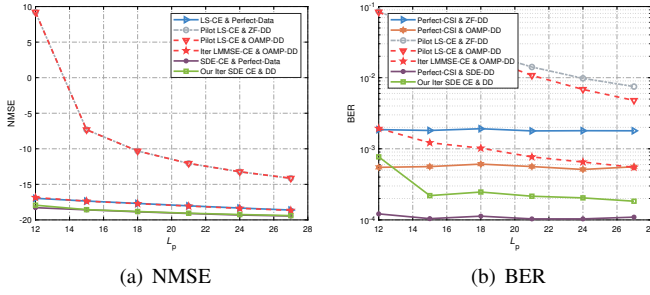


Fig. 9: NMSE and BER performance with respect to the pilot symbol length  $L_p$  for different CE and DD algorithms.

of **SDE-CE & Perfect-Data**, whereas the LS algorithm, which relies solely on pilots, exhibits a notable performance gap relative to iterative CE and DD algorithms. This suggests that additional computation can potentially reduce communication pilot overhead. Also **Iter LMMSE-CE & OAMP-DD** and **LS-CE & Perfect-Data** exhibit similar performance, indicating that the estimated data is sufficiently robust to enhance CE. Fig. 9(b) shows that the BER performance of **Our Iter SDE CE & DD** is considerably better than that of **Perfect-CSI & OAMP-DD**, which relies on the perfect CSI. As expected, **Perfect-CSI & SDE-DD** attains the best performance.

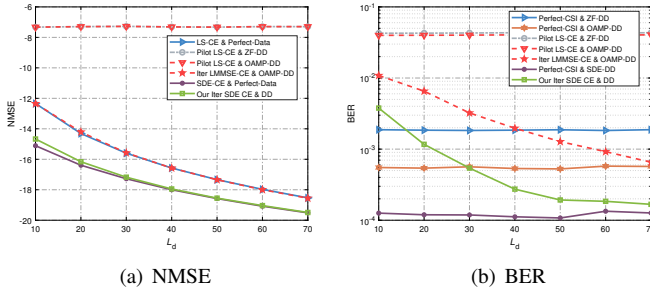


Fig. 10: NMSE and BER performance with respect to the data symbol length  $L_d$  for different CE and DD algorithms.

Fig. 10 depicts the performance curves of CE and DD across various schemes as the functions of data length  $L_d$ . In the simulation, the pilot length for each UE is fixed at  $L_p = 15$ , with the received SNR=10dB at the BS. As can be seen from Fig. 10(a), the iterative algorithm improves CE performance as data length increases. For  $L_d \geq 30$ , the NMSE of **Our Iter SDE CE & DD** matches that of **SDE-CE & Perfect-Data**, offering an NMSE gain of 1 to 2 dB over **Iter LMMSE-CE & OAMP-DD**. As depicted in Fig. 10(b), increasing  $L_d$  improves the accuracy of CE, thereby enhancing the BER performance of **Iter LMMSE-CE & OAMP-DD** and **Our Iter SDE CE & DD**. Moreover, when  $L_d = 70$ , the BER of **Iter LMMSE-CE & OAMP-DD** approximates that of **Perfect-CSI & OAMP-DD**, while **Our Iter SDE CE & DD** surpasses **Perfect-CSI & OAMP-DD**, in terms of BER.

Fig. 11 compares the performance of CE and DD of **Our Iter SDE CE & DD** scheme as the function of the algorithm iteration numbers  $N_H$  and  $N_X$ . In the simulation, each UE's pilot length is set to  $L_p = 15$  and the data length is  $L_d = 50$ , with the received SNR fixed at 10dB. For the predefined noise diffusion functions  $\sigma(t)$  and  $\tau(t)$ , the interval  $t \in [0, 1]$  can be divided into segments with different values of  $N_X$  and  $N_H$  to control the noise variance gap between adjacent steps,

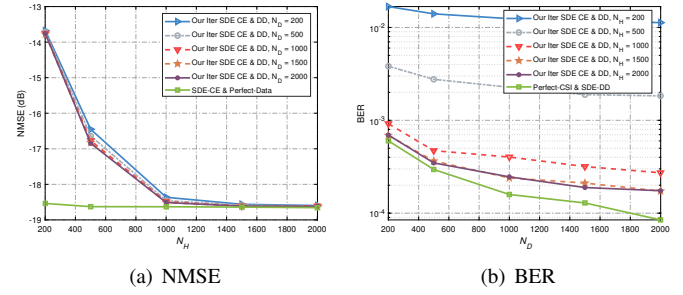


Fig. 11: NMSE and BER performance with respect to the step numbers  $N_H$  and  $N_X$  of our Iter SDE CE & DD algorithm.

thereby balancing generation speed and performance. As shown in Fig. 11(a), as the channel sampling steps  $N_H$  increase, the NMSE performance improves gradually due to a more elaborate denoising process, resulting in more accurate SDE solutions. When  $N_H = 1000$ , the NMSE of **Our Iter SDE CE & DD** approximates that of **SDE-CE & Perfect-Data**. Moreover, under the same  $N_H$ , CE performance remains similar for different  $N_D$ , possibly due to SDE-based CE not being sensitive to DD errors. Additionally, Fig. 11(b) shows that as the data sampling steps  $N_D$  increase, the BER performance of DD gradually improves. However, the BER curves vary for different  $N_H$ , indicating that for DD problems, SDE-based DD methods are more sensitive to CE errors.

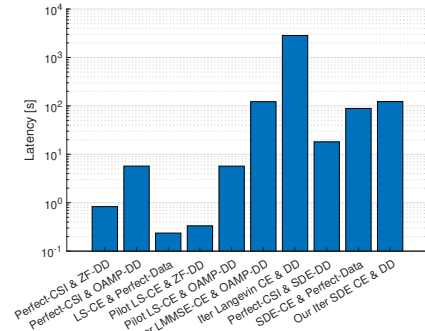


Fig. 12: Computation latency of different CE and DD schemes.

Fig. 12 compares the computational latency of various methods for processing a batch size of 64 in CE and DD tasks, utilizing a NVIDIA GeForce GTX 4090 GPU and an AMD EPYC 7J13 CPU. Due to its asynchronous update mechanism and LMMSE initialization, **Our Iter SDE CE & DD** algorithm demonstrates significantly lower computational latency compared to **Iter Langevin CE & DD** algorithm. Furthermore, the latency of **Our Iter SDE CE & DD** is comparable to that of **Iter LMMSE-CE & OAMP-DD**, while the previous simulations have shown it outperforming **Iter LMMSE-CE & OAMP-DD** in both CE and DD. Although the latency of iterative CE and DD algorithms is higher than that of two-stage algorithms, such as **Pilot LS-CE & ZF-DD** and **Pilot LS-CE & OAMP-DD**, existing researches indicate that diffusion-based algorithms can reduce the number of steps from thousands to tens, or even to as few as 1 to 2 steps, through acceleration methods such as higher-order Langevin algorithms [26] and consistency models [42]. Therefore, accelerating diffusion algorithms still holds significant potential for addressing CE and DD problems.

## VII. CONCLUSIONS

We have proposed techniques for addressing the challenges of AUD and JCEDD in massive random access, where standard deep learning often struggles to generalize across various problem dimensions using a single model. Our VPL-AUDNet architecture has been designed for variable pilot length transmission, enabling a single model to adapt to different pilot lengths and antennas, thereby achieving superior AUD performance in both 3GPP and Rayleigh channels. Furthermore, we have introduced a generative diffusion models-driven JCEDD scheme using PC samplers, which significantly enhances CE and DD performance through an asynchronous iterative algorithm. These proposed schemes have demonstrated the potential of utilizing deep learning and generative models to address signal detection and estimation challenges in massive random access. A future research direction of interest is to consider accelerating generative model methods to further reduce inference latency.

## REFERENCES

- [1] Z. Gao, *et al.*, “Compressive-sensing-based grant-free massive access for 6G massive communication,” *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7411–7435, Mar. 2024.
- [2] ITU-R WP 5D, “The ITU-R Framework for IMT-2030,” July 2023.
- [3] X. Shao, *et al.*, “Cooperative activity detection: Sourced and unsourced massive random access paradigms,” *IEEE Trans. Signal Process.*, vol. 68, pp. 6578–6593, Dec. 2020.
- [4] Z. Chen, F. Sohrabi, and W. Yu, “Sparse activity detection in multi-cell massive MIMO exploiting channel large-scale fading,” *IEEE Trans. Signal Process.*, vol. 69, pp. 3768–3781, Jul. 2021.
- [5] W. Jiang, Y. Jia, and Y. Cui, “Statistical device activity detection for OFDM-based massive grant-free access,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3805–3820, Jun. 2023.
- [6] Y. Li, *et al.*, “Heterogeneous transformer: A scale adaptable neural network architecture for device activity detection,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 3432–3446, May 2023.
- [7] Z. Sheng, P. Zhu, and X. You, “UADFormer: A transformer-based deep learning method for user activity detection in cell-free massive MIMO systems,” *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 18516–18531, Dec. 2024.
- [8] L. Qiao, *et al.*, “Compressive sensing based massive access for IoT relying on media modulation aided machine type communication,” *IEEE Trans. Veh. Technol.*, vol. 69, no. 9, pp. 10391–10396, Sep. 2020.
- [9] M. Ke, *et al.*, “Compressive sensing-based adaptive active user detection and channel estimation: Massive access meets massive MIMO,” *IEEE Trans. Signal Process.*, vol. 68, pp. 764–779, Feb. 2020.
- [10] K. Ying, *et al.*, “Quasi-synchronous random access for massive MIMO-based LEO satellite constellations,” *IEEE J. Sel. Areas Commun.*, vol. 41, no. 6, pp. 1702–1722, Jun. 2023.
- [11] Y. Cui, S. Li, and W. Zhang, “Jointly sparse signal recovery and support recovery via deep learning with applications in MIMO-based grant-free random access,” *IEEE J. Sel. Areas Commun.*, vol. 39, no. 3, pp. 788–803, Mar. 2021.
- [12] X. Shao, *et al.*, “Feature-aided adaptive-tuning deep learning for massive device detection,” *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 1899–1914, Jul. 2021.
- [13] J. Ma and L. Ping, “Orthogonal AMP,” *IEEE Access*, vol. 5, pp. 2020–2033, Mar. 2017.
- [14] J. Céspedes, P. M. Olmos, M. Sánchez-Fernández, and F. Perez-Cruz, “Expectation propagation detection for high-order high-dimensional MIMO systems,” *IEEE Trans. Commun.*, vol. 62, no. 8, pp. 2840–2849, Aug. 2014.
- [15] K. Pratik, B. D. Rao, and M. Welling, “RE-MIMO: Recurrent and permutation equivariant neural MIMO detection,” *IEEE Trans. Signal Process.*, vol. 69, pp. 459–473, Jan. 2021.
- [16] H. He, C.-K. Wen, S. Jin, and G. Y. Li, “Model-driven deep learning for MIMO detection,” *IEEE Trans. Signal Process.*, vol. 68, pp. 1702–1715, Mar. 2020.
- [17] A. Kosasih, *et al.*, “Graph neural network aided expectation propagation detector for MU-MIMO systems,” *IEEE J. Sel. Areas Commun.*, vol. 40, no. 9, pp. 2540–2555, Sep. 2022.
- [18] H. He, *et al.*, “Message passing meets graph neural networks: A new paradigm for massive MIMO systems,” *IEEE Trans. Wireless Commun.*, vol. 23, no. 5, pp. 4709–4723, May 2024.
- [19] S. Zhang, Y. Cui, and W. Chen, “Joint device activity detection, channel estimation and signal detection for massive grant-free access via BiGAMP,” *IEEE Trans. Signal Process.*, vol. 71, pp. 1200–1215, Apr. 2023.
- [20] M. Ke, *et al.*, “Next-generation URLLC with massive devices: A unified semi-blind detection framework for sourced and unsourced random access,” *IEEE J. Sel. Areas Commun.*, vol. 41, no. 7, pp. 2223–2244, Jul. 2023.
- [21] M. Arvinte and J. I. Tamir, “MIMO channel estimation using score-based generative models,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3698–3713, Jun. 2023.
- [22] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” in *Adv. Neural Inf. Process. Syst.*, Vancouver, Canada, Dec. 8–14, 2019, pp. 11918–11930.
- [23] Y. Song and S. Ermon, “Improved techniques for training score-based generative models,” in *Adv. Neural Inf. Process. Syst.*, Dec. 6–12, 2020, pp. 12438–12448.
- [24] L. He, Z. Wang, and Y. Huang, “A massive MIMO sampling detection strategy based on denoising diffusion model,” in *IEEE Int. Wireless Commun. Mobile Comput. (IWCMC)*, Ayia Napa, Cyprus, May 27–31, 2024, pp. 1443–1448.
- [25] N. Zilberstein, *et al.*, “Annealed Langevin dynamics for massive MIMO detection,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3762–3776, Jun. 2023.
- [26] N. Zilberstein, A. Sabharwal, and S. Segarra, “Solving linear inverse problems using higher-order annealed Langevin diffusion,” *IEEE Trans. Signal Process.*, vol. 72, pp. 492–505, Jan. 2024.
- [27] N. Zilberstein, A. Swami, and S. Segarra, “Joint channel estimation and data detection in massive MIMO systems based on diffusion models,” in *IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Seoul, South Korea, Apr. 14–19, 2024, pp. 13291–13295.
- [28] X. Zhou, *et al.*, “Generative diffusion models for high dimensional channel estimation,” *IEEE Trans. Wireless Commun.* (Early Access), Mar. 2025. DOI:10.1109/TWC.2025.3549592
- [29] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Adv. Neural Inf. Process. Syst.*, Dec. 6–12, 2020, pp. 6840–6851.
- [30] B. Fesl, *et al.*, “Diffusion-based generative prior for low-complexity MIMO channel estimation,” *IEEE Wireless Commun. Lett.*, vol. 13, no. 12, pp. 3493–3497, Dec. 2024.
- [31] T. Wu, *et al.*, “CDDM: Channel denoising diffusion models for wireless semantic communications,” *IEEE Trans. Wireless Commun.*, vol. 23, no. 9, pp. 11168–11183, Sep. 2024.
- [32] Y. Song, *et al.*, “Score-based generative modeling through stochastic differential equations,” in *Int. Conf. Learn. Represent.*, May 3–7, 2021, pp. 1–36.
- [33] 3GPP, “Study on channel model for frequencies from 0.5 to 100 GHz,” 3rd Generation Partnership Project (3GPP), Technical Report (TR) 38.901, 05 2017, version 14.0.0.
- [34] J. Xu, *et al.*, “Wireless image transmission using deep source channel coding with attention modules,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 4, pp. 2315–2328, Apr. 2022.
- [35] S. H. Chan, “Tutorial on diffusion models for imaging and vision,” Jan. 2025. [Online] Available: <https://arxiv.org/abs/2403.18103>.
- [36] B. D. O. Anderson, “Reverse-time diffusion equation models,” *Stochastic Processes and their Applications*, vol. 12, no. 3, pp. 313–326, May 1982.
- [37] B. Kassar, M. Elad, S. Ermon, and J. Song, “Denoising diffusion restoration models,” in *Adv. Neural Inf. Process. Syst.*, Nov. 28–Dec. 9, 2022, pp. 23593–23606.
- [38] P. Vincent, “A connection between score matching and denoising autoencoders,” *Neural Comput.*, vol. 23, no. 7, pp. 1661–1674, Jul. 2011.
- [39] X. Meng and Y. Kabashima, “Quantized compressed sensing with score-based generative models,” in *Int. Conf. Learn. Represent.*, Kigali, Rwanda, May 1–5, 2023, pp. 1–30.
- [40] B. Efron, “Tweedie’s formula and selection bias,” *J. Am. Stat. Assoc.*, vol. 106, no. 496, pp. 1602–1614, Jan. 2011.
- [41] S. Jaecel, *et al.*, “QuaDRiGa-quasi deterministic radio channel generator, user manual and documentation,” Heinrich Hertz Institute, Tech. Rep. v2.6.1, 2021.
- [42] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *Int. Conf. Mach. Learn.*, Honolulu, HI, USA, Jul. 23–29, 2023, 2023, pp. 32211–32252.